Middleware Parameters

Additionally, middleware can be given more arguments. For instance, you might design an **`EnsureUserHasRole`** middleware that accepts a role name as a second parameter if your application has to confirm that the authenticated user has a specific "role" before carrying out a specific operation.

After the **`$next`** argument, the middleware will receive additional parameters:

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;
class EnsureEmployeeHasRole
{
 /**
  * Handle an incoming request.
  *
  * @param \Closure(\Illuminate\Http\Request):
  (\Symfony\Component\HttpFoundation\Response) $next
  */
 public function handle(Request $request, Closure $next, string
 $role): Response
 {
  if (! $request->employee()->hasRole($role)) {
   // Redirect Logic
  }
  return $next($request);
 }
}
```

When specifying the route, middleware parameters may be supplied by separating the middleware name and parameters with a `:`. Comma-separated lists of arguments should be used:

```
Route::put('/post/{emp_id}', function (string $emp_id) {
 // Code Logic
})->middleware('role:editor')
```